

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS • Minikube
Built	11 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 3 — State, Configuration and Data

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 3

What changed today

Yesterday	Today
Config hard-coded in eleven manifests	Two ConfigMaps
JWT signing key in a plain env var	A Secret, mounted read-only
Every payment lost on pod restart	PostgreSQL with 5,000 transactions
Fraud counters per-replica	Shared Redis counters
No storage at all	PV, PVC, StorageClass, three StatefulSets
Containers run as root	UID 10001, read-only rootfs, zero capabilities

3.1 ConfigMaps

1. What it is

A named collection of key/value pairs, stored in the cluster, that pods can read as environment variables or as files.

2. Why it exists

Configuration changes far more often than code. Baking it into a workload definition means a code review and a pipeline run to change a log level.

3. The business problem

AxisPay's fraud team wants the review threshold lowered from 45 to 40 for the weekend. Today that edits `fraud-service`'s Deployment — a change to the payment path — for a number. Meanwhile `LOG_LEVEL` is duplicated across eleven manifests, so turning on debug logging during an incident means eleven edits at 03:00.

4. How it works

Two consumption modes, and they behave completely differently:

Mode	Behaviour	Consequence
<code>env / envFrom</code>	Values copied into the process environment once , at container creation	A snapshot . Changing the ConfigMap does nothing to a running pod — no error, no event.
<code>volumeMounts</code>	Each key projected as a file, updated in place by the kubelet	Changes appear within ~60 s. But Kubernetes updated the <i>file</i> ; the application must re-read it.

The `subPath` exception. Mounting a single key with `subPath` produces a real file rather than a symlink into the projected directory — and it **never updates**. This is documented, and it is the most common "why is my mounted config not changing" question.

5. Internal architecture

A ConfigMap is a plain object in etcd. For volume mounts the kubelet writes the data into a `tmpfs` directory on the node and symlinks the keys into it. Updates are atomic: the kubelet writes a new timestamped directory and swings the symlink, so a reader never sees a half-written file.

Size limit: **1 MiB**, inherited from etcd's practical object limit. AxisPay's 10 KiB schema fits; the 8 MB seed file does not, which is why it is piped into `psql` instead.

6. Component interactions

```

you           kubectl apply configmap
API server    stores in etcd
kubelet       (env)   reads once at container create – snapshot
kubelet       (volume) watches; on change writes a new dir and swings the symlink
your app      (volume) must re-read the file to notice
  
```

7. Enterprise example

A payments platform keeps one ConfigMap per environment holding endpoints, timeouts and feature flags, and a separate one for risk thresholds mounted as a volume. Application config is a snapshot (deliberate, releasable); risk tuning is live (deliberate, operational). The split is a design decision recorded in the runbook, not an accident.

8. Real-world analogy

An env var is a **printed** copy of the price list handed to a cashier at the start of a shift. A mounted volume is a **screen** on the wall showing the current list — but the cashier still has to look up.

9. Best practices

Practice	Reason
Use env for values read once at startup	A snapshot is honest about how the value is used
Use a volume for values operations must change live	No rollout on the payment path
Add a checksum annotation to the pod template	Config changes become releases: visible in <code>rollout history</code> , rollback-able
Never put credentials in a ConfigMap	It has none of a Secret's (already limited) protections
Keep ConfigMaps small	1 MiB limit; large ones bloat every watch
Consider <code>immutable: true</code> at scale	Removes the kubelet's watch and reduces API server load

10. Common mistakes

Mistake	Symptom
Expecting env vars to update live	Nothing changes; nothing warns you
Using <code>subPath</code> and expecting updates	The file never changes
Key renamed	Env var silently absent, app falls back to a default — this is INC-3b
Credentials in a ConfigMap	Readable by anyone with <code>get configmaps</code>
ConfigMap over 1 MiB	Rejected at creation

11. Security considerations

Threat	Control	Residual risk
Credentials placed in a ConfigMap	Policy + review; use Secrets	Secrets are barely better without RBAC
ConfigMap readable cluster-wide	RBAC per namespace (Day 5)	Namespace admins can read all of them
Config injection changing behaviour	Restrict <code>update</code> on ConfigMaps in production	A compromised CI account can still change them

12. Performance considerations

Each mounted ConfigMap adds a watch per pod. At thousands of pods this becomes measurable API server load — which is what `immutable: true` addresses. Volume updates are not instant: they follow the kubelet sync period, typically up to 60 seconds.

13. High availability

ConfigMaps are metadata and inherit control-plane HA. Note the failure mode: if a ConfigMap is deleted, **running pods keep working** (env vars are already copied, volumes already projected), but **new pods fail to start** with `CreateContainerConfigError`. The damage appears at the next scale-up or rollout, not immediately.

14. Disaster recovery

Trivially reproducible from manifests — provided they are in Git. A ConfigMap edited live with `kubectl edit` and never committed is exactly the drift that makes a rebuild fail.

15. Monitoring

Signal	Why
<code>CreateContainerConfigError</code> events	Missing ConfigMap or key
Pods not restarted after a config change	The checksum annotation is missing
ConfigMap size approaching 1 MiB	Will be rejected

16. Troubleshooting

Symptom	Cause	Command	Fix
CreateContainerConfigError	ConfigMap or key missing	<code>kubectl describe pod</code> → Events	Create it; check the key name
Env var not updating	Snapshot	—	Restart, or use a volume
Mounted file not updating	<code>subPath</code> , or <code>immutable</code>	<code>kubectl get cm -o yaml</code>	Remove <code>subPath</code>
App reads a default instead of your value	Key name mismatch	<code>kubectl exec -- printenv \ sort</code>	Compare with the ConfigMap

Interview questions

- You change a ConfigMap. Do running pods see it?** *Depends how it is consumed. As env vars, no — they are a snapshot from container creation. As a mounted volume, yes, within about 60 seconds — but the application must re-read the file.*
- How do you make a config change trigger a rollout?** *Put a hash of the ConfigMap in a pod-template annotation. Any change alters the template, creating a new ReplicaSet and rolling the pods. Helm does this automatically.*
- Why is there a 1 MiB limit?** *(senior) It comes from etcd's practical object size. It matters in practice: schema files fit, seed data does not, and large ConfigMaps are replicated to every watching kubelet.*
- When would you set `immutable: true`?** *(senior) At scale. It tells the kubelet not to watch the object, cutting API server load significantly across thousands of pods. The cost is that a change requires creating a new ConfigMap and updating the reference — which some teams consider a feature, because it makes config changes explicit releases.*

3.2 Secrets

1. What it is

An object for holding small pieces of sensitive data — passwords, tokens, keys — that pods consume as environment variables or files.

2. Why it exists

To keep credentials out of images and manifests, and to give the cluster a distinct object type that RBAC and encryption-at-rest can be applied to.

3. The business problem

Since Day 1 the JWT signing key has been a plain environment variable in `auth-service`'s Deployment. Anyone with `get pods` in `axispay-edge` can read it with `kubectl describe`. With that key they can mint a valid token for **any** merchant and authorise payments as them.

4. How it works

A Secret is base64-encoded. It is not encrypted.

```
kubectl get secret axispay-db-credentials -n axispay-data \
-o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d
```

There is no key, no algorithm, nothing to break. `base64 -d` is a formatting change. Anyone who can read the object can read every value in it.

What actually protects a Secret:

Control	What it does
RBAC	Restricts who may <code>get</code> Secrets, per namespace. <i>The real boundary.</i>
Encryption at rest	etcd encrypts Secret values on disk. Without it, an etcd backup is a plaintext credential dump.
Mount as a volume, not env	Fewer accidental leak paths (see §9)
External secret manager	Short-lived, rotatable, audited credentials. The production answer.

5. Internal architecture

Secret volumes are mounted as **tmpfs** — RAM-backed, never written to the node's disk. Files default to mode `0644`; AxisPay sets `0400`. Like ConfigMaps, updates are projected atomically.

`stringData` accepts plaintext in a manifest and Kubernetes base64-encodes it on write. It is more readable than pre-encoded `data:` and exactly as (in)secure.

6. Component interactions

Identical to ConfigMaps, with one difference: the kubelet only fetches Secrets a pod actually references, and holds them in memory rather than on disk.

7. Enterprise example

A bank runs External Secrets Operator against HashiCorp Vault. Kubernetes Secrets exist, but are **projections** with a 1-hour TTL, refreshed automatically. Nothing sensitive is ever committed to Git; the manifest contains only a reference. Rotation is a Vault operation and the cluster follows within the hour.

8. Real-world analogy

A Secret is a **sealed envelope in an unlocked filing cabinet**. The envelope stops casual glancing; the lock on the cabinet (RBAC) is what actually protects it. Calling the envelope "encryption" is the mistake.

9. Best practices

Practice	Reason
Prefer volumes over env vars	Env vars are inherited by children, appear in <code>/proc/*/environ</code> , and are captured by crash handlers
Set <code>defaultMode: 0400</code>	Read-only, owner only
Never commit a Secret manifest to Git	Use sealed-secrets or an external manager
Enable etcd encryption at rest	Otherwise a backup is a credential dump
One Secret per concern	Limits blast radius and makes RBAC meaningful
Support two valid keys during rotation	Otherwise rotation is an outage
Restrict <code>get secrets</code> tightly	This is the actual control (Day 5)

10. Common mistakes

Mistake	Symptom
Believing Secrets are encrypted	Credentials exposed in an etcd backup
Using a plain <code>value:</code> instead of <code>secretKeyRef</code>	Visible in <code>kubectl describe pod</code>
Committing Secret manifests	Credentials in Git history forever
Trailing newline from <code>--from-file</code>	Auth fails with a correct-looking password
Rotating without a restart	Env vars are snapshots
One giant Secret for everything	Any pod needing one value gets all of them

11. Security considerations

Threat	Control	Residual risk
<code>kubectl get secret</code> by an over-privileged user	RBAC	Cluster-admin can always read them
etcd backup stolen	Encryption at rest + KMS	Key management becomes the problem
Env var captured by an error tracker	Mount as a volume	Deliberate exfiltration still possible
Secret in Git history	Sealed secrets / external manager	History rewriting is painful
Node compromise	tmpfs limits disk exposure	An attacker with root on the node can read process memory

The honest summary: today's work is a real improvement — reading the JWT key now needs `get secrets` as well as `get pods`. It is not encryption, and the boundary is RBAC.

12. Performance considerations

Negligible. Secrets are small by design (1 MiB limit). The only consideration at scale is the same watch load as ConfigMaps.

13. High availability

Same as ConfigMaps: deleting a Secret does not break running pods, but new pods fail with `CreateContainerConfigError`. The failure surfaces at the next rollout.

14. Disaster recovery

Secrets are the one thing you cannot restore from Git, because they should not be in Git. Recovery means an external secret manager, a sealed-secrets controller with its private key backed up separately, or a documented manual re-creation procedure. Deciding which — before an incident — is the actual DR work.

15. Monitoring

Signal	Why
Audit log: <code>get secrets</code> by unexpected principals	Credential access is worth alerting on
<code>CreateContainerConfigError</code>	Missing Secret
Secret age	Rotation overdue

16. Troubleshooting

Symptom	Cause	Command	Fix
CreateContainerConfigError	Secret or key missing	<code>kubectl describe pod</code>	Create it; check the key
Auth fails with a correct password	Trailing newline	<code>... \ base64 -d \ xxd \ tail -1</code>	Use <code>--from-literal</code> or <code>printf</code>
Permission denied reading a mounted secret	<code>defaultMode</code> vs <code>runAsUser</code>	<code>kubectl exec -- ls -la</code>	Align <code>defaultMode</code> and <code>fsGroup</code>
Rotation had no effect	Env var snapshot	—	<code>kubectl rollout restart</code>
Value visible in <code>describe</code>	Plain <code>value:</code> used	<code>kubectl get deploy -o yaml</code>	Use <code>secretKeyRef</code>

Interview questions

- 1. Is a Kubernetes Secret encrypted?** No. *base64-encoded, which is an encoding, not encryption. What protects it is RBAC on who may read the object, plus etcd encryption at rest so a backup is not a plaintext credential dump.*
- 2. Why prefer a volume over an environment variable?** *Env vars are inherited by every child process, appear in `/proc/<pid>/environ`, and are routinely captured whole by crash handlers and error-tracking SDKs. A file is read once by the code that needs it, and Secret volumes are tmpfs so they never touch the node's disk.*
- 3. How would you rotate a signing key with no downtime?** (senior) *The application must accept two valid keys during a rotation window: add the new key, deploy so both are trusted, switch signing to the new one, wait for old tokens to expire, then remove the old key. Kubernetes can deliver the material but cannot make the rotation safe — that is an application design decision.*
- 4. How do you keep Secrets out of Git while still doing GitOps?** (senior) *Sealed Secrets (encrypt with a controller's public key; only the in-cluster controller can decrypt) or External Secrets Operator (Git holds a reference; the operator fetches from Vault/AWS/GCP and projects a short-lived Secret). The second is generally preferred because rotation is handled outside the cluster.*

3.3 The storage model

1. What it is

A set of objects that let a pod use storage which outlives it: **Volume** (ephemeral), **PersistentVolume** (a real piece of storage), **PersistentVolumeClaim** (a request for one), **StorageClass** (a recipe for making them on demand).

2. Why it exists

A container's filesystem is part of the container. Delete the pod and it is gone. Anything with a memory needs storage decoupled from the pod lifecycle.

3. The business problem

AxisPay must retain transaction records for seven years and reconcile them nightly. You cannot reconcile against memory, and a Python dictionary is not a system of record.

4. How it works

The **claim/bind contract** separates supply from demand:

```
Storage admin (or a provisioner) -> PersistentVolume "here is 2Gi, RW0, Retain"
Developer -> PersistentVolumeClaim "I need 1Gi, RW0"
Control plane -> BIND if capacity, access mode and class all agree
Pod -> mounts the CLAIM, never the volume
```

That indirection is why the same manifest runs on Minikube hostPath, AWS EBS and a corporate SAN unchanged.

Access modes — the most misread field in Kubernetes:

Mode	Means	Does mean
ReadWriteOnce	One NODE may mount it read-write	~~one pod~~ — pods on the <i>same node</i> can share it
ReadOnlyMany	Many nodes, read-only	
ReadWriteMany	Many nodes, read-write. Needs NFS/CephFS.	Most block storage physically cannot do this
ReadWriteOncePod	Genuinely one pod	The mode people usually meant by RWO

ReadWriteOncePod exists *because* the misreading above was so common.

Reclaim policy — what happens to the DATA when the claim is deleted:

Policy	Effect	Right for
Delete	Volume and data destroyed	Caches, scratch, CI
Retain	PV goes Released, data kept, manual reclaim	Anything you cannot recreate

5. Internal architecture

CSI splits into two halves:

- **Controller plugin** — one per cluster. Talks to the storage API: "create a 5Gi volume". A cloud API call.
- **Node plugin** — a DaemonSet on every node. Attaches, formats and mounts. A kernel operation on one machine.

They are separate because creating a volume and mounting it are completely different operations in different places.

`volumeBindingMode` decides *when* binding happens:

Mode	Sequence	Result
<code>Immediate</code>	Volume created on some node → scheduler later picks a node → they may disagree	<code>volume node affinity conflict</code> — pod unschedulable forever
<code>WaitForFirstConsumer</code>	PVC stays Pending → scheduler picks a node → volume created there	Storage and scheduling agree by construction

On a multi-node cluster with node-local storage this is not an optimisation; it is the difference between working and not.

6. Component interactions

```
PVC created          -> provisioner watches
(WaitForFirstConsumer) -> nothing yet; PVC stays Pending
pod created          -> scheduler filters and scores, picks a node
provisioner          -> creates the volume ON THAT NODE, creates the PV, binds
CSI node plugin      -> attaches and mounts into the pod's namespace
kubelet              -> applies fsGroup (chowns) so a non-root process can write
```

7. Enterprise example

A bank uses a CSI driver against its SAN, with three StorageClasses: `gold` (replicated NVMe, `Retain`), `silver` (SSD, `Retain`), `bronze` (spinning, `Delete`). Developers request a class by name; the storage team owns what each means. Nobody writes a `PersistentVolume` by hand.

8. Real-world analogy

A PVC is a **hotel booking**: "one double room, three nights". The PV is the actual room. You never book room 412 — you book a *kind* of room and the system assigns one. A StorageClass is the rate plan that decides which rooms you can be assigned and what happens at checkout.

9. Best practices

Practice	Reason
Retain for anything you cannot recreate	A deleted PVC must not destroy a ledger
WaitForFirstConsumer on node-local storage	Prevents the affinity conflict
Always set fsGroup for non-root workloads	Otherwise permission denied on a fresh volume
Never write PVs by hand in production	Use dynamic provisioning
Request the size you need, not "a bit extra"	Expansion is easy; shrinking is impossible
allowVolumeExpansion: true	Growing later without recreating
Give hostPath PVs nodeAffinity	Or the pod may schedule where the data is not

10. Common mistakes

Mistake	Symptom
Reading RWO as "one pod"	Designing around a guarantee you do not have
Requesting RWX on block storage	PVC Pending, terse event
Delete on a data volume	PVC deleted, data gone
No fsGroup	<code>permission denied</code> on a directory that looks fine
Immediate binding, multi-node	<code>volume node affinity conflict</code>
PGDATA at the mount root	<code>initdb: directory not empty (lost+found)</code>
Expecting a PVC to shrink	Not supported

11. Security considerations

Threat	Control	Residual risk
hostPath exposes the node filesystem	Forbid hostPath via Pod Security Admission	System workloads legitimately need it
Data at rest unencrypted	Encrypted StorageClass parameters	Key management
Deleted PVC destroys evidence	Retain + RBAC on <code>delete pvc</code>	Cluster-admin can still delete
Volume reused with old data	Retain requires explicit reclaim	Manual step can be done carelessly

12. Performance considerations

- **Node-local (hostPath) is fastest and least available.** Networked storage is the reverse.
- IOPS and throughput usually come from StorageClass **parameters**, not from Kubernetes.
- **WaitForFirstConsumer** adds a small delay to first start — worth it.
- Many small volumes cost more control-plane work than a few large ones.

13. High availability

Node-local storage pins a pod to a node. If that node is unavailable, the pod cannot run anywhere — and Kubernetes correctly refuses to start it elsewhere, because starting a database on an empty volume looks exactly like data loss.

Real answers, in order: networked storage that any node can mount · database-level replication so another replica holds the data · accepting the pinning and planning maintenance around it.

14. Disaster recovery

Back up	The volume contents — Kubernetes does not back up your data
How	Volume snapshots (<code>VolumeSnapshot</code>), or application-level dumps (<code>pg_dump</code>) shipped off-cluster
RPO	Your snapshot interval
RTO	Restore + replay time; test it
The trap	A PV is not a backup. <code>Retain</code> protects against an accidental delete, not against corruption, and not against a failed disk.

15. Monitoring

Metric	Threshold
<code>kube_persistentvolumeclaim_status_phase{phase="Pending"}</code>	> 2 min — this is INC-3a
<code>kubelet_volume_stats_available_bytes / capacity_bytes</code>	< 15%
<code>kube_persistentvolume_status_phase{phase="Failed"}</code>	Any
Released PVs accumulating	<code>Retain</code> needs manual reclaim

16. Troubleshooting

Symptom	Cause	Command	Fix
PVC Pending, "waiting for first consumer"	Normal	<code>kubectl describe pvc</code>	Create a pod
PVC Pending, "storageclass not found"	Typo or missing class	<code>kubectl get sc</code>	Fix <code>storageClassName</code> — INC-3a
PVC Pending, no matching PV	Size/mode/class mismatch	<code>kubectl describe pvc</code>	Align them
volume node affinity conflict	Volume on another node	<code>kubectl describe pod</code>	<code>WaitForFirstConsumer</code> , or networked storage
permission denied on the volume	No <code>fsGroup</code>	<code>kubectl exec -- ls -ld</code>	Set <code>fsGroup</code>
PV Released, will not rebind	Retain keeps <code>claimRef</code>	<code>kubectl get pv -o yaml</code>	Clear <code>claimRef</code> or recreate

Interview questions

1. What does **ReadWriteOnce** actually mean? One node may mount it read-write. Several pods on that same node can share it. `ReadWriteOncePod` is the mode that genuinely means one pod, and it exists because RWO was so widely misread.
2. Why would you delay binding a volume? `WaitForFirstConsumer` lets the scheduler choose the node first, then provisions the volume there. With `Immediate` the volume can be created on a node the scheduler will not use, and the pod is unschedulable forever with `volume node affinity conflict`.
3. **Retain versus Delete**? `Delete` destroys the volume and its data when the claim goes; right for caches and CI. `Retain` keeps it in a `Released` state for manual recovery; right for anything you cannot recreate. It is a decision made once in the `StorageClass`, and it is the difference between a recoverable mistake and a destroyed ledger.
4. Your PVC has been Pending for five minutes. Broken or working as intended? (senior) Read the event. "waiting for first consumer" is correct behaviour — create a pod. "storageclass not found" or "no persistent volumes available" is genuinely broken. The status is identical; the event is everything.
5. Is a **PersistentVolume a backup**? (senior) No. It protects against pod loss, not against corruption, deletion or disk failure. Backups need snapshots or application-level dumps shipped off-cluster — and a tested restore. `Retain` is a safety catch, not a backup strategy.

3.4 StatefulSets

1. What it is

A controller for workloads where each replica has an identity: a stable name, its own storage, and a defined start/stop order.

2. Why it exists

Deployments treat pods as interchangeable. Databases, queues and consensus systems need the opposite of that in all three dimensions.

3. The business problem

AxisPay needs PostgreSQL. A Deployment gives random names, shared-or-no storage, and replacement in any order — which produces two unrelated empty databases behind a load balancer.

4. How it works

Three guarantees a Deployment cannot provide:

Guarantee	Mechanism	Why it matters
Stable name	Ordinal naming: <code>postgres-0</code> , <code>postgres-1</code>	A replica can be told "your primary is <code>postgres-0</code> " and that stays true
Stable storage	<code>volumeClaimTemplates</code> → one PVC per replica, re-attached by ordinal	<code>postgres-0</code> always gets <code>data-postgres-0</code>
Stable order	<code>podManagementPolicy: OrderedReady</code> — start 0,1,2; terminate 2,1,0	A replica never starts before its primary

The **headless Service** (`clusterIP: None`) makes DNS return pod IPs rather than one virtual IP, giving `postgres-0.postgres.axispay-data.svc.cluster.local`. You cannot address one specific replica through a normal Service.

5. Internal architecture

`volumeClaimTemplates` creates PVCs named `<template>-<statefulset>-<ordinal>`. **These are not deleted when the StatefulSet is deleted** — deliberately, because deleting a workload must never silently destroy its data.

`podManagementPolicy` options: `OrderedReady` (default, sequential) or `Parallel` (all at once — right for workloads with no ordering requirement, such as a sharded cache).

6. Component interactions

```

StatefulSet controller -> creates PVC data-postgres-0
                        -> creates pod postgres-0, waits for Ready
                        -> creates PVC data-postgres-1, pod postgres-1 ...
headless Service       -> DNS: postgres-0.postgres.<ns>.svc.cluster.local
on reschedule          -> SAME name, SAME PVC, new node (if storage allows)

```

7. Enterprise example

Nobody runs a production PostgreSQL cluster on a bare StatefulSet. They use an **Operator** (CloudNativePG, Zalando, Crunchy) which builds on StatefulSets and adds what Kubernetes does not know: replication, leader election, failover, WAL archiving, backups. The StatefulSet provides identity and storage; the Operator provides the database.

8. Real-world analogy

A Deployment is a **pool of temps** — any of them will do. A StatefulSet is **named staff with assigned desks**: Desk 0 is always the same person's desk, their belongings are still in it when they return, and the induction order is fixed.

9. Best practices

Practice	Reason
Always pair with a headless Service	<code>serviceName</code> must match, or pods never get stable DNS
Never <code>Delete</code> reclaim policy	The PVCs are the data
Generous startup probes	<code>initdb</code> , recovery and schema load take time
Long <code>terminationGracePeriodSeconds</code>	Let a database checkpoint and close cleanly
Set <code>fsGroup</code>	Non-root databases cannot write to a fresh volume otherwise
Use an Operator for real clusters	Kubernetes knows nothing about replication
Readiness must run a real query	<code>pg_isready</code> succeeds while the server is still in recovery

10. Common mistakes

Mistake	Symptom
Database as a Deployment	Random names, lost data, no ordering
<code>serviceName</code> does not match	Pods never get stable DNS
Scaling to 3 and expecting a cluster	Three independent empty databases
Deleting the StatefulSet expecting cleanup	PVCs remain (deliberate)
<code>PGDATA</code> at the mount root	<code>initdb: directory not empty</code>
Missing <code>fsGroup</code>	Permission denied on a fresh volume
Aggressive liveness on a database	Restart during recovery, restart loop

11. Security considerations

Threat	Control	Residual risk
Database reachable from the whole cluster	NetworkPolicy (Day 4)	Needs a policy-enforcing CNI
Database running as root	<code>runAsNonRoot</code> + correct UID (postgres uses 999)	Image must support it
Credentials in the pod spec	Secret + <code>envFrom</code>	Secrets are not encrypted
Volume readable on the node	Encrypted storage class	Node compromise

12. Performance considerations

- **Ordered startup is serial.** Ten replicas mean ten sequential starts. `Parallel` is available when ordering does not matter.
- Storage class choice dominates database performance far more than anything Kubernetes does.
- Rolling updates go in reverse ordinal order, one at a time — safe, and slow.

13. High availability

A **StatefulSet is not high availability**. It gives identity and storage; it does not replicate data. Three PostgreSQL replicas in a StatefulSet are three separate databases unless something above them implements replication.

Real HA needs: an Operator managing replication and failover · anti-affinity so replicas are on different nodes (Day 4)
 · a PodDisruptionBudget (Day 4) · storage that survives node loss.

14. Disaster recovery

The PVCs are the data. Recovery means restoring volume snapshots or database dumps — Kubernetes objects alone restore an **empty** database with the right name.

Test the restore. A `pg_dump` that has never been restored is not a backup.

15. Monitoring

Metric	Threshold
<code>kube_statefulset_status_replicas_ready</code> vs <code>_replicas</code>	Gap > 5 min
<code>kubelet_volume_stats_available_bytes</code>	< 15%
Pod restart count on a data workload	Any — investigate
<code>kube_statefulset_status_observed_generation</code> lag	Controller not reconciling

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>postgres-1</code> never starts	<code>OrderedReady</code> , pod 0 not Ready	<code>kubectl get pods</code>	Fix pod 0 first
Pod Pending after a node drain	Node-local volume elsewhere	<code>kubectl describe pod</code>	Uncordon, or networked storage
PVCs remain after deletion	Deliberate	<code>kubectl get pvc</code>	Delete explicitly if you mean it
<code>initdb: directory not empty</code>	<code>PGDATA</code> at the mount root	<code>kubectl logs</code>	Use a subdirectory
Permission denied on the data dir	No <code>fsGroup</code>	<code>kubectl exec -- ls -ld</code>	Set <code>fsGroup: 999</code>
Restart loop during recovery	Liveness too aggressive	<code>kubectl describe pod</code>	Longer thresholds; add a startup probe

Interview questions

- 1. Name three things a StatefulSet gives that a Deployment cannot.** *Stable network identity (ordinal names plus per-pod DNS via a headless Service), stable per-replica storage (`volumeClaimTemplates` , re-attached by ordinal), and ordered startup and termination.*
- 2. Why does a StatefulSet need a headless Service?** *`clusterIP: None` makes DNS return pod IPs rather than a single virtual IP, so `postgres-0.postgres.<ns>.svc.cluster.local` resolves to that specific pod. A normal Service load-balances and cannot address one replica.*
- 3. You scale a PostgreSQL StatefulSet to 3. Do you have a cluster?** *No. Three independent empty databases. Kubernetes provides identity, storage and ordering; it knows nothing about replication, leader election or WAL shipping. A real cluster needs an Operator.*
- 4. Why are PVCs not deleted with the StatefulSet?** *(senior) Because deleting a workload must never silently destroy its data. It is a deliberate safety default; cleanup is explicit. Newer Kubernetes offers `persistentVolumeClaimRetentionPolicy` if you genuinely want automatic deletion.*
- 5. Your database pod is Pending after a node was drained. Why, and is Kubernetes wrong?** *(senior) Node-local storage pins the volume to one node. Kubernetes is refusing to start the database somewhere its data does not exist — which is correct. Starting it on an empty volume would look exactly like data loss. The fix is networked storage or database-level replication, not overriding the scheduler.*

3.5 securityContext

1. What it is

Fields on a pod or container that constrain what the process may do: which user it runs as, whether it can write to its filesystem, and which Linux capabilities it holds.

2. Why it exists

A container is a process on a shared kernel. If it runs as root and escapes, it is root on the node — and on that node's other workloads.

3. The business problem

`axispay-core` is the cardholder data environment. A container escape there is not an incident, it is a breach notification.

4. How it works

Setting	Level	Effect
<code>runAsNonRoot: true</code>	pod	The kubelet refuses to start the pod if the image's user resolves to UID 0
<code>runAsUser / runAsGroup</code>	pod/ container	The UID/GID the process runs as
<code>fsGroup</code>	pod	The kubelet chowns mounted volumes to this GID before start
<code>readOnlyRootFilesystem</code>	container	Root filesystem mounted read-only
<code>capabilities.drop: ["ALL"]</code>	container	Removes the ~14 capabilities containers get by default
<code>allowPrivilegeEscalation: false</code>	container	Blocks <code>setuid</code> binaries from gaining privileges
<code>seccompProfile: RuntimeDefault</code>	pod	Blocks ~44 dangerous syscalls

`fsGroup` is the one that catches everyone. A freshly provisioned volume is owned by root. A process running as UID 10001 cannot write to it, and the error points at a directory that looks perfectly normal.

5. Internal architecture

These map onto Linux primitives: user namespaces and UID/GID, cgroups, capability bounding sets, seccomp BPF filters, and a read-only bind mount for the root filesystem. Kubernetes is a configuration surface over kernel features that already existed.

`capabilities.drop: ["ALL"]` removes `NET_RAW` — enough to craft raw packets and ARP-spoof other pods on the node. A payment API needs none of the default capabilities.

6. Component interactions

admission	validates the spec (Pod Security Admission – Day 5)
kubelet	resolves the image user; REFUSES if runAsNonRoot and user is 0
kubelet	chowns volumes per fsGroup
runtime	applies capability bounding set, seccomp, read-only rootfs

7. Enterprise example

A bank enforces `restricted` Pod Security Admission on every namespace holding cardholder data. A workload without a compliant `securityContext` is **rejected at admission** — it never runs. Exceptions require a documented, time-boxed waiver. Defence in depth: images set `USER`, pod specs enforce it, and the namespace policy rejects anything that does neither.

8. Real-world analogy

Building access. **Defence in depth**: the employee has the right badge (the image sets `USER`), the door checks it (the pod spec enforces `runAsNonRoot`), and the floor is access-controlled regardless (Pod Security Admission). Any one alone can be circumvented; all three rarely.

9. Best practices

Practice	Reason
Set <code>USER</code> in the image and <code>runAsNonRoot</code> in the spec	Images regress; manifests get copied
<code>readOnlyRootFilesystem: true</code> + an <code>emptyDir</code> for <code>/tmp</code>	A compromised process cannot write a payload
<code>capabilities.drop: ["ALL"]</code> , add back only what is needed	Almost nothing needs any
<code>allowPrivilegeEscalation: false</code> always	Blocks <code>setuid</code> escalation
<code>fsGroup</code> matching the runtime GID	Or non-root cannot write to volumes
Use ports above 1024	Avoids needing <code>NET_BIND_SERVICE</code>
Enforce at namespace level	Day 5 — so forgetting is not possible

10. Common mistakes

Mistake	Symptom
Only setting <code>USER</code> in the image	A future build regresses it silently
<code>readOnlyRootFilesystem</code> with no writable temp	App crashes writing a temp file
Forgetting <code>fsGroup</code>	<code>permission denied</code> on a fresh volume
Wrong UID for the image	Postgres uses <code>999</code> , not <code>10001</code>
Dropping capabilities a workload needs	<code>Operation not permitted</code> binding port 80
Assuming non-root is enough	<code>setuid</code> binaries can still escalate

11. Security considerations

Threat	Control	Residual risk
Container escape as root	<code>runAsNonRoot</code> + drop ALL + seccomp	Kernel vulnerabilities remain
Malware written to disk	<code>readOnlyRootFilesystem</code>	tmpfs is still writable
Privilege escalation via setuid	<code>allowPrivilegeEscalation: false</code>	—
Packet sniffing / ARP spoofing	Drop <code>NET_RAW</code>	A privileged sidecar could reintroduce it
Host filesystem access	Forbid <code>hostPath</code> (Day 5)	Some agents legitimately need it

12. Performance considerations

Essentially free. `seccompProfile: RuntimeDefault` adds a small per-syscall filter cost, immaterial for a network service. `fsGroup` chowns the volume at mount — on a very large volume with millions of files that can be slow, which is what `fsGroupChangePolicy: OnRootMismatch` addresses.

13. High availability

No direct effect. Indirectly relevant: a compromised container can degrade or destroy a node's other workloads, so hardening limits blast radius.

14. Disaster recovery

`securityContext` is configuration and recovers with the manifest. The DR-relevant point is a **compromised image**: recovery means rebuilding from a known-good base and rotating every credential the workload could reach — which is far easier if the workload could only reach a few.

15. Monitoring

Signal	Why
Pods rejected for <code>runAsNonRoot</code>	Something is trying to run as root
Pod Security Admission violations (Day 5)	Policy drift
Audit log: pods with <code>privileged: true</code>	Should be a very short list
Read-only filesystem write errors	Missing <code>emptyDir</code>

16. Troubleshooting

Symptom	Cause	Command	Fix
container has runAsNonRoot and image will run as root	Image has no USER	<code>docker inspect</code> <code>\ grep User</code>	Add USER 10001
Read-only file system	Writing outside a mount	<code>kubectl logs</code>	<code>emptyDir</code> for that path
permission denied on a volume	No fsGroup	<code>kubectl exec --</code> <code>ls -ld</code>	Set fsGroup
Operation not permitted on port 80	NET_BIND_SERVICE dropped	<code>grep CapEff /</code> <code>proc/1/status</code>	Use a port above 1024
Postgres will not start non-root	Wrong UID	<code>kubectl logs</code>	The postgres image uses 999

Interview questions

1. What does `fsGroup` do? It makes the kubelet chown mounted volumes to that GID before the container starts, so a non-root process can write to a freshly provisioned volume. Without it you get permission denied on a directory that looks fine.
2. Why enforce non-root in the pod spec when the image already sets `USER` ? Defence in depth. A future image build can regress, a base image can change, and a manifest can be copied without its `securityContext` . With both, a careless change is caught by one of them.
3. Why does `allowPrivilegeEscalation: false` matter if you already run as non-root? (senior) Because a `setuid` binary inside the container can gain privileges the parent process did not have. Running as UID 10001 does not prevent `execve` of a `setuid-root` binary; this flag sets `no_new_privs` , which does.
4. What does `capabilities.drop: ["ALL"]` actually remove? (senior) The ~14 capabilities containers get by default — including `NET_RAW` (raw packets, ARP spoofing), `CHOWN` , `SETUID` , `SETGID` and `MKNOD` . A typical web service needs none of them. Verify with `grep CapEff /proc/1/status` ; hardened should read all zeroes.

Day 3 cheat sheet

Config and secrets

```
kubectl get configmap,secret -n axispay-core
kubectl describe configmap axispay-platform-config -n axispay-core

# decode a Secret – base64 is ENCODING, not encryption
kubectl get secret axispay-db-credentials -n axispay-data \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d; echo

# env vars are a SNAPSHOT – a change needs a restart
kubectl rollout restart deployment/<name> -n <ns>

# mounted volumes update in place (~60s) – but subPath NEVER does
kubectl exec -n <ns> <pod> -- cat /etc/axispay/rules/fraud-rules.yaml
kubectl exec -n <ns> <pod> -- mount | grep secrets    # tmpfs
```

Storage

```
kubectl get sc,pv,pvc -A
kubectl describe pvc <name> -n <ns>      # the EVENT is everything

# is Pending normal or broken?
#   "waiting for first consumer"      -> normal, create a pod
#   "storageclass ... not found"      -> broken
#   "no persistent volumes available" -> broken

kubectl get pv -o custom-columns=\
NAME:.metadata.name,CAP:.spec.capacity.storage,\
RECLAIM:.spec.persistentVolumeReclaimPolicy,STATUS:.status.phase,CLAIM:.spec.claimRef.name
```

Access mode	One...
ReadWriteOnce	NODE (not pod)
ReadWriteOncePod	pod, genuinely
ReadWriteMany	needs NFS/CephFS — most block storage cannot

StatefulSets

```
kubectl get statefulset,pvc -n axispay-data
kubectl get svc postgres -n axispay-data          # CLUSTER-IP must be None

# stable per-pod DNS
nslookup postgres-0.postgres.axispay-data.svc.cluster.local

# PVC naming: <template>-<statefulset>-<ordinal>
#   data-postgres-0, data-postgres-1 ...
# NOT deleted when the StatefulSet is deleted – deliberate.
```

Database

```
kubectl exec -it -n axispay-data postgres-0 -- psql -U axispay_app -d axispay
```

```
SELECT COUNT(*) FROM payments;                -- 5000
SELECT * FROM v_ledger_balance;                -- imbalance must be 0
SELECT COUNT(*) FROM payments WHERE amount_minor <> fee_minor + net_minor; -- 0
SELECT currency, COUNT(*), SUM(amount_minor)/100.0 FROM payments GROUP BY 1;
```

securityContext

```
kubectl exec -n <ns> <pod> -- id                # uid=10001
kubectl exec -n <ns> <pod> -- grep CapEff /proc/1/status # 0000000000000000
kubectl exec -n <ns> <pod> -- sh -c 'touch /x'    # Read-only file system
```

Control	Prevents
<code>runAsNonRoot</code>	Escape becoming root on the node
<code>readOnlyRootFilesystem</code>	Writing a payload to disk
<code>capabilities.drop: ALL</code>	Raw packets, ARP spoofing, setuid, mknod
<code>allowPrivilegeEscalation: false</code>	setuid escalation
<code>fsGroup</code>	<i>(not security — makes volumes writable)</i>

Day 3 review questions

1. Change a ConfigMap. Do running pods see it? Both cases.
2. Why does a `subPath` mount never update?
3. Is a Kubernetes Secret encrypted? What actually protects it?
4. Name three ways a Secret can leak, and a control for each.
5. Why prefer a Secret volume over an environment variable?
6. What does `ReadWriteOnce` actually mean?
7. Which access mode genuinely guarantees a single writer?
8. Explain `WaitForFirstConsumer` by describing the failure it prevents.
9. `Retain` vs `Delete` — which for a payments ledger, and why?
10. What does `fsGroup` do, and what breaks without it?
11. Name three guarantees a StatefulSet gives that a Deployment cannot.
12. Why does a StatefulSet need a headless Service?
13. You scale PostgreSQL to 3 replicas. Do you have a cluster?
14. Why are PVCs not deleted with the StatefulSet?
15. Why is `PGDATA` a subdirectory of the mount point?
16. Why enforce non-root in the pod spec when the image sets `USER` ?
17. What does `capabilities.drop: ["ALL"]` remove, and how do you verify it?
18. Your PVC is Pending. How do you tell "normal" from "broken"?
19. Is a PersistentVolume a backup?
20. Why is the `payments_balance` CHECK in the schema rather than in the application?

Answers: <documents/assessments/answer-keys/day3-answer-key.md>

Day 3 summary

You built: two ConfigMaps replacing config in eleven manifests · five Secrets with the JWT key out of plaintext · a StorageClass with `Retain` and `WaitForFirstConsumer` · a static PV/PVC pair · three StatefulSets with persistent volumes · **5,000 payments and a ledger that balances to zero across 4,955 journals** · init containers for ordered startup · every application pod at UID 10001 with zero capabilities.

You proved: env vars are snapshots and volumes update in place · a Secret decodes in one command · a database as a Deployment loses its data and its identity · node-local storage pins a pod to a node, and Kubernetes correctly refuses to start it elsewhere · `fsGroup` is what makes a non-root workload able to write.

And you fixed yesterday's bug. Fraud velocity counters are in Redis, so the threshold means what it says at any replica count.

What is still missing:

Gap	Consequence	Fixed
No route in from outside	No merchant can integrate	L4.3
No TLS	Card data over plaintext HTTP	L4.3
Flat pod network	<code>axispay-edge</code> can reach PostgreSQL directly — a PCI finding	L4.4
No placement control	Three payment replicas can land on one node	L4.5
No disruption budget	A node drain takes every replica at once	L4.6
DNS still unopened	You have relied on it for three days	L4.2

Tonight (optional, 10 minutes): exec into an `edge-gateway` pod and connect to PostgreSQL directly. It works. Understanding why that is a finding — and not a feature — is the whole of Day 4.